Reproducibility and Numerical Conditioning of the Drain-Refined Impact-Ionization Trio

Liang–1997 SHE–BTE 0.35 μm LDD n-MOSFET reproduction

Technical report for independent review

2026-10-16 (frozen solver tag `solver-frozen-2026-09-26`, git `930dd86`)

Executive summary

The certified reproduction of Liang *et al.* (1997) yields a drain-refined spatial-convergence trio of the peak impact-ionization generation rate G_{ii} at three meshes (71×58, 100×70, 141×85 on a common frozen energy grid). This report documents a multi-run, multi-environment reproducibility campaign of that trio. Two findings:

1. **The qualitative verdict is robust and reproduces everywhere tested**: the trio is significantly *non-monotone* ($S_{\text{rev}} \gg 1$), so Richardson extrapolation is suppressed and **there is no justified continuum value of** G_{ii}. This holds across every Windows run.
2. **The absolute peak G_{ii} is *not* reproducible.** It moves with the convergence/gate-trip point ($\sim 2\%$), the compute kernel (numba vs. pure-Python, $\sim$ 3–7%), the SciPy version ($\sim$ 3%), and – most strongly – the **operating system** ($\sim 12\%$ Windows$\leftrightarrow$Linux with *identical* code, Python, NumPy and SciPy versions). The SciPy-version hypothesis was tested and *disproved*: matching the Linux library stack to the Windows reference made the discrepancy *worse* ($-9.1\% \to -12.0\%$).

Peak G_{ii} is a near-null-space, high-energy-tail functional of the distribution, hypersensitive to the platform numerical floating-point path. A staged frozen-state probe (Appendix C) localizes the *first* cross-platform divergence to sparse-system assembly (matrix coefficients differ at $\sim 10^{-16}$, single-threaded), upstream of the sparse solve; a single frozen-state solve reproduces to $\sim 4 \times 10^{-13}$. Whether that rounding-scale assembly seed, amplified over the nonlinear iteration, fully *accounts* for the -12% full-run difference is plausible but not yet proved (see Appendix C, "what is not yet established"). The Sep-24 reference was computed on Windows and reproduces bit-exactly only on Windows. The recommended remedy is to report a well-conditioned integrated observable (and a properly-defined terminal substrate current) rather than a mesh-point maximum.

1 System, reference, and method

First-order spherical-harmonic expansion (SHE) of the electron Boltzmann equation in total-energy form, self-consistent with 2D Poisson and hole continuity; nonparabolic Kane band; DOS-weighted acoustic + optical-phonon + Keldysh-surrogate impact-ionization operators. The SHE linear system is solved with LGMRES preconditioned by a physics-split incomplete-LU (`scipy.sparse.linalg.spilu`); the outer map is Anderson-accelerated (depth = 6, $\beta = 0.5$). Convergence gate: last-4 cumulative-iteration spread of $G_{ii,\max}$ and $G_{tot} \leq 3 \times 10^{-4}$ and outer residual $R_\phi \leq 1.5 \times 10^{-3}$.

Certified reference (Sep-24, Windows, pure-Python kernel), produced by git `d30b471`, validated equivalent to the frozen tag:

$G_{ii} = 3.075619/2.136180/2.387429 \times 10^{25}\,\mathrm{cm^{-3}s^{-1}}$, $S_{\mathrm{rev}} = 737$, non-monotone, Richardson suppressed.

Common frozen H-grid hash `b4319bb03303` (591 energy points) across all meshes and all runs.

2 Runs and environments

run	machine	kernel	library stack
Sep-24 reference	Windows, 24 GB	pure-Python	NumPy 2.5.1 / SciPy 1.18.0 / Py 3.13
Windows interrupted	Windows	pure-Python	same (run broken by ∼4 OOM resumes)
Windows clean	Windows (64 GB pagefile)	pure-Python	same, uninterrupted
Linux numba	Linux, 31 GiB, 24 thr	**numba JIT**	NumPy 2.2.6 / SciPy 1.15.3
Linux pure-Python #1	Linux	pure-Python	NumPy 2.2.6 / SciPy 1.15.3
Linux pure-Python #3	Linux	pure-Python	**NumPy 2.5.1 / SciPy 1.18.0 / Py 3.13.12 (ma**

3 Results

3.1 Per-node converged G_{ii} ($\times 10^{25}\,\mathrm{cm^{-3}s^{-1}}$)

node	Sep-24 ref	Win intr.	Win clean	Linux numba	Lx py 1.15	Lx py 1.18
71×58	3.075619	3.075619	3.075619[†]	2.9847	2.7955	2.7067[‡]
100×70	2.136180	2.142249	2.087561	∼1.80[§]	—	—
141×85	2.387429	2.375921	2.399434	—	—	—

[†]reused the earlier clean-run checkpoint (an independent clean pure-Python derivation reproduced 3.075619 to 1.6×10^{-7}, bit-exact). [‡]did *not* certify (spread stuck $\sim 9 \times 10^{-3}$, G_{ii} declining); killed at 360 iters ≈ the reference's 359. [§]hotspot limit-cycle, never certified.

3.2 The qualitative verdict reproduces

run	sequence signs	monotone?	S_{rev}
Sep-24 reference	$(-,+)$	no	737
Windows interrupted	$(-,+)$	no	1018
Windows clean	$(-,+)$	no	2415

Every completed trio is significantly non-monotone; no real convergence order exists, Richardson extrapolation is suppressed, and G_{ii} stays bounded in $[2.14, 3.08] \times 10^{25}$. **This conclusion is robust to environment, kernel, and interruption.**

3.3 The non-reproducibility hierarchy (one variable at a time)

variable changed (all else fixed)	isolated comparison	ΔG_{ii}
convergence / gate-trip point (Win, pure-Py)	100×70: 2.0876 vs 2.1362	−2.3%
compute kernel (Linux, SciPy 1.15)	71×58: numba 2.9847 vs pure-Py 2.7955	+6.8%
SciPy version (Linux, pure-Py)	71×58: 1.15.3 (2.7955) vs 1.18.0 (2.7067)	−3.2%
operating system (pure-Py, matched versions)	71×58: Win 3.0756 vs Linux 2.7067	**−12.0**%

Measured against the Windows reference, the Linux 71×58 lands at −2.9% (numba/1.15), −9.1% (pure-Py/1.15), and −12.0% (pure-Py/1.18 matched).

4 The decisive platform test

Hypothesis under test: the Linux discrepancy is caused by the SciPy version. Prediction: pinning Linux to the reference stack (NumPy 2.5.1 / SciPy 1.18.0, Python 3.13.12, numba off) should recover $\sim 3.0756 \times 10^{25}$.

Outcome: the opposite. On the matched stack, 71×58 plateaued at 2.7067×10^{25} (-12.0%) and failed to certify. The 60-iteration trajectory diverges from the reference *at the first chunk*:

chunk	cum iters	G_{ii} ($\times 10^{25}$)	last-4 spread	gate
0	60	2.5476	1.25×10^{-2}	cont
1	120	2.5622	4.02×10^{-3}	cont
2	180	2.6420	6.52×10^{-3}	cont
3	240	2.7120	9.15×10^{-3}	cont
4	300	2.7105	7.67×10^{-3}	cont
5	360	2.7067	9.42×10^{-3}	cont (killed; G_{ii} declining)

Reference chunk-0 (Windows) is 2.5571×10^{25}; Linux chunk-0 is 2.5476×10^{25} – already different at iteration 60. Because the matched run *increased* the gap, SciPy version is excluded as the cause. The only remaining difference between the Windows reference and Linux #3 is the **operating system** and its compiled numerical libraries (BLAS/LAPACK/SuperLU, SIMD). The run also fails to converge cleanly on Linux: the pointwise-maximum G_{ii} node jitters between iterations, so the last-4 spread never approaches the 3×10^{-4} gate.

5 Mechanism — why it fails

Peak G_{ii} is numerically ill-posed, for three compounding reasons:

1. **Tail-dominated functional.** G_{ii} integrates the distribution's high-energy tail ($\varepsilon \gtrsim 1.5\,\mathrm{eV}$), many orders of magnitude below the peak of f. It is therefore controlled by a tiny, ill-conditioned part of the solution – the documented "near-null-space": a 10^{-8} *linear* residual does not pin G_{ii}.
2. **Floating-point-path-dependent solves.** LGMRES + `spilu` results depend on summation and pivoting order, which differ between BLAS/LAPACK/SuperLU builds (OS, compiler, SIMD) at the $\sim 10^{-15}$ level per operation.
3. **Amplification by the fixed-point iteration.** Over ~ 350 outer iterations these $\sim 10^{-15}$ differences compound, and because G_{ii} lies in the ill-conditioned direction, they are amplified into $\sim 10\%$ in the converged peak – and can shift which plateau the iteration selects, or prevent clean convergence (the hotspot limit-cycle).

Hence identical code and library versions on a different OS give a -12% peak G_{ii}. The same mechanism explains why the numba kernel, the SciPy version, and the gate-trip point each move the value: all perturb the same ill-conditioned direction. The maximum over mesh nodes of a tail quantity is not a numerically well-posed observable.

6 Remediation

These are proposed methodology changes to the frozen reference; none has been applied.

- **Report an integrated observable, not a mesh-point maximum (primary fix).** Quote an integrated generation $\int G_{ii}\, dV$ (well-conditioned, averages out the hotspot floating-point noise) and, for a *terminal* figure, the properly-defined substrate hole current – the hole flux collected at the body contact including recombination (audit finding F-08). Note $\int G_{ii}\, dV$ is the generation *rate* per out-of-plane width, not by itself the collected terminal current; both are far better conditioned than the mesh-point maximum. This is standard TCAD practice.
- **Direct linear solve.** Replace Krylov+ILU with a direct LU factorization where memory permits (raise the existing direct-solve DOF cutoff), removing the preconditioner/Krylov floating-point-path dependence.
- **Gate hardening.** Require plateau flatness sustained over multiple chunks with small chunk-to-chunk ΔG_{ii}, and gate on the integrated G_{ii}, so "converged" means the same thing on every platform.
- **Report with a numerical band.** Absolute peak G_{ii} carries a $\sim 10\%$ numerical uncertainty; report it as a band $[2.1, 3.1] \times 10^{25}$ with the stated cause – which is precisely the existing non-asymptotic (YELLOW) conclusion.

7 For the independent reviewer

- **Robust (verify on Windows):** the non-monotone trio and Richardson suppression – stable across all Windows runs and both kernels/versions.
- **Not portable (verify):** absolute peak G_{ii}, which differs $\sim 12\%$ Windows↔Linux with matched code and versions. A minimal confirmation: run one SHE assembly+solve from an identical stored state on both OSs and diff the output vector element-wise (expected $\sim 10^{-12}$–10^{-15} seed).
- **Preserved evidence:** per-chunk metrics CSVs, residual histories, run logs, the H-hash, the environment manifests, and the certified `reference_sep24/` NPZs are in the repository (https://github.com/chanlikt-lgtm/Liang1997).

Appendix: provenance

Solver tag `solver-frozen-2026-09-26` (git `930dd86`); reference git `d30b471`. Windows: Python 3.13.15, NumPy 2.5.1, SciPy 1.18.0. Linux #3: Python 3.13.12 (uv venv), NumPy 2.5.1, SciPy 1.18.0, numba absent (pure-Python assembly confirmed by ∼10 s/assembly vs. ∼0.2 s numba). Frozen H-grid hash `b4319bb03303`, 591 energy points, identical across all meshes and runs. No thermal throttling in the final Linux run (package 70–82 °C).

Appendix B: Direct platform-FP probe (one-shot, isolated)

The identical 71×58 cold-start coupled solve was run for 3 capped outer iterations on both machines (`probe_fp.py`) and the outputs diffed element-wise. The mesh and energy grid are built from element-wise NumPy and are **bit-identical** across the two operating systems; the only variable is the platform numerical stack.

Environment. Both ran NumPy 2.5.1 / SciPy 1.18.0 / Python 3.13.x, pure-Python assembly, and – notably – the *same OpenBLAS version 0.3.33*. Windows: 8 threads, pthreads. Linux: DYNAMIC_ARCH, runtime SIMD kernel X86_V3 (AVX2), ∼8 cores.

Result.

quantity	max $\|\Delta\|$/scale	max pointwise rel. diff
mesh x, y, energy grid H (input)	0 (bit-identical)	0
ϕ (potential, well-conditioned)	2.7×10^{-13}	2.7×10^{-11}
G_{ii} (near-null-space)	1.6×10^{-10}	$\mathbf{6.6 \times 10^{-4}}$

The peak-G_{ii} divergence *grows per outer iteration*:

iter 0: 5.7×10^{-13} $\rightarrow$ iter 1: 2.2×10^{-12} $\rightarrow$ iter 2: 1.5×10^{-10} ($\sim$260$\times$ per 3 iterations).

Interpretation (refined by Appendix C). Identical input and identical library *version* still diverge. This probe reran the full coupled initialization on each machine, so it establishes the *sensitivity and amplification* but not the first-divergence *stage*; the staged probe of Appendix C localizes the seed to assembly-coefficient floating point (confirmed single-threaded), *upstream* of the sparse solve – superseding an earlier guess of BLAS reduction order. The mechanism is unchanged: a $\sim 10^{-16}$ platform floating-point seed, amplified by the ill-conditioning of G_{ii}, yields an order-10% non-portable result.

Appendix C: Staged causal localization (frozen-input probe)

Prompted by an independent source audit, a fixed-state probe (`audit/fixed_state_probe.py`) froze ONE 71×58 state (mesh, ϕ, contact map, equilibrium density n_{eq}) from the Windows reference, copied the *identical bytes* to both operating systems, and – single-threaded (`OPENBLAS_NUM_THREADS=1`), pure-Python – assembled the SHE operator and solved it on each.

quantity	Windows vs. Linux
state x, y, ϕ, contact map, n_{eq}	**bit-identical** (SHA-256 match)
energy grid H, active-state map	**bit-identical**
CSR structure A.`indptr`, A.`indices`	**bit-identical**
A.`data` (matrix coefficients)	**DIFFER** (max rel 1.1×10^{-16}, last ULP)
b (right-hand side)	**DIFFER** (agree to 16 digits)
single solve from frozen state: G_{ii}^{peak}	3.0756185×10^{25} both (rel diff 3.8×10^{-13})

Localization (what is established). Cross-platform differences *first become observable during sparse-system assembly*: the CSR structure, energy grid and active map are bit-identical, but the matrix coefficients (A.`data`) and right-hand side (b) differ at approximately floating-point rounding precision ($\sim 10^{-16}$, last ULP), and this holds *single-threaded* – so it is not BLAS thread-reduction order. The likely source is the platform's vectorized elementary functions (`exp`/`pow` in the DOS-weighted Fokker–Planck, phonon and impact-ionization coefficients) differing between OS/compiler builds of NumPy/libm. A single frozen-state solve nevertheless reproduces peak G_{ii} across OS to $\sim 4{\times}10^{-13}$ relative accuracy, so SuperLU/LGMRES is *not* the primary seed. The positivity clipping (audit finding F-02) is benign on both OSs (clipped and raw G_{ii} peaks identical; tail-negative ℓ_1 mass 1.5×10^{-14}).

What is NOT yet established. That the full-run -12% is *caused* by this $\sim 10^{-16}$ assembly seed amplified over $\sim$ 360 iterations is *plausible but not proved.* The larger full-run differences are *consistent with* amplification during nonlinear iteration, but the present experiment does not distinguish three possibilities: (i) assembly rounding is the dominant source; (ii) other platform-dependent operations also contribute per iteration; (iii) the nonlinear map has strong sensitivity or multiple attracting trajectories. The precise amplification mechanism and the contribution of other platform-dependent operations *remain to be established.*

Decisive next tests (pre-registered). (1) *Single-platform perturbation*: on one machine, perturb the assembly coefficients by $\sim 10^{-16}$ relative magnitude each iteration and run the full loop from the same checkpoint with several random seeds; if these consistently reach *different* plateaus, that is direct evidence of nonlinear sensitivity to a rounding-scale seed. (2) *Coefficient replay* (strongest): drive the Linux iteration with the Windows-computed coefficients (or a shared portable elementary-function path); if the long-run divergence is *suppressed*, assembly rounding is confirmed as the dominant source; if it persists, other platform-dependent operations contribute. These are the strongest tests before declaring the root cause closed.

Test 1, first result (single-platform sensitivity). Controlled perturbation experiments demonstrate that rounding-scale modifications to the assembly coefficients can produce rapidly amplified differences in the early coupled iteration trajectory. Injecting a 10^{-16} relative random perturbation each outer iteration on Windows (runtime monkeypatch, no source edit; of the intended perturbation, $\sim$42% of A.`data` entries and $\sim$1% of b entries survive floating-point rounding, applied RMS $\sim 1.2 \times 10^{-16}$) yields relative peak-G_{ii} deviations from the unperturbed control of $\sim 6.7 \times 10^{-13}$, $\sim 9 \times 10^{-13}$, $\sim 1.7 \times 10^{-10}$ at iterations 0–2 – *comparable in magnitude* to the observed early Windows–Linux divergence (5.7×10^{-13}, 2.2×10^{-12}, 1.5×10^{-10}); two independent seeds yield distinct trajectories of the same magnitude. These results establish *short-term* nonlinear sensitivity to assembly-level floating-point perturbations. Three iterations do not establish an amplification law, and random 10^{-16} noise is not equivalent to the structured, sign-correlated actual Windows–Linux coefficient differences. Whether this sensitivity explains the $\sim 12\%$ long-run discrepancy remains under investigation through (a) certified full-trajectory perturbation ensembles and (b) cross-platform coefficient-replay experiments (driving one platform's iteration with the other's *measured* coefficients; one-shot and across the trajectory). The most likely scientific outcome is a *characterization of the nonlinear solver's reproducibility limits* for this near-null-space quantity, not the identification of a single defective numerical library.